

v2 チェック



Thought for 1m 14s >

結論から言うと、「できているか？」は **UI** を眺めても分からないので、

V2.pdf を「仕様書 兼 受け入れテスト一覧」として **モジュールごとに検証用テストを用意する**のが一番筋がいいです。 V2

1. 全体の考え方（何を確認すべきか）

V2 では大きく 4 つのモジュールが定義されています V2

1. Multi-Arm Causal Design Studio（多アーム因果推論）
2. Experiment Orchestrator & Bandit Control Tower（オンライン配信最適化）
3. Growth & LTV Planning Studio（LTV・リテンション）
4. Risk & Governance Center（フェアネス・品質ゲート）

確認したいことは **3 点**だけです：

1. 仕様に書かれている **API / クラス / UI 要素が存在するか**
2. 単純なおもちゃデータで **数学的におかしくない値が出ているか**
3. 既存 v1 の機能を **壊していないか**

これをモジュール A～D それぞれについて、

- ユニットテスト（Python）
- 簡易 E2E（UI から期待値が見えるか）

で押さえます。

2. テストの置き場と実行コマンド

まず、バックエンド側に「V2 検証用テスト」をまとめるディレクトリを切ってしまいます。

```
# backend 側
cd backend

# V2 用テストディレクトリを作成
mkdir -p tests/v2

# 既にあるならスキップされるのでそのまま実行してOK
```

pytest 実行コマンドはシンプルにこれだけにしておくと良いです。

```
cd backend
pytest tests/v2 -q
```

フロントの E2E を Playwright/Cypress でやるなら、後で

```
cd frontend
npx playwright test # or: npx cypress run
```

のように「E2E 用コマンド」を 1 本決めておきます。

3. モジュール A - Multi-Arm Causal Design Studio の確認

3.1 仕様と実装の存在確認

V2.pdf では `MultiArmDREstimator` のクラス骨格が示されています V2

- `backend/engine/estimators/multi_arm_dr.py` が存在するか
- クラス `MultiArmDREstimator` に `fit(X, T, Y)` / `effect(X)` があるか
- Causal Design UI 上部に「Treatment Type: Binary / Multi-Armed / Dose-Response」トグルがあるか

をまず人間の目で確認。

3.2 ユニットテスト例（多アームが動くか）

```
# backend/tests/v2/test_multi_arm_dr.py
import numpy as np
from backend.engine.estimatedors.multi_arm_dr import MultiArmDREstimator

def make_toy_data(n=3000, seed=0):
    rng = np.random.default_rng(seed)
    X = rng.normal(size=(n, 2))
    # 真の効果: arm1 = +1, arm2 = +2
    T = rng.integers(0, 3, size=n) # 0, 1, 2
    base = X[:, 0] * 0.0
    tau1, tau2 = 1.0, 2.0
    tau = (T == 1) * tau1 + (T == 2) * tau2
    Y = base + tau + rng.normal(scale=0.5, size=n)
    return X, T, Y, tau1, tau2

def test_multi_arm_effect_order():
    X, T, Y, tau1, tau2 = make_toy_data()

    est = MultiArmDREstimator(base_learner=lambda: ExistingDRLearner())
    est.fit(X, T, Y)
    eff = est.effect(X) # dict: arm -> tau_hat

    tau1_hat = eff[1].mean()
    tau2_hat = eff[2].mean()

    # arm2 の方が arm1 より大きいことだけをチェック
    assert tau2_hat > tau1_hat
    # オーダーが完全に崩れていないことも軽く見る
    assert 0.2 < tau1_hat < tau2_hat < 3.0
```

What this does (このテストがすること)

- **ゴール:** 多アーム推定器 `MultiArmDREstimator` が、「効果が大きいアームをちゃんと大きく推定できているか」を最低限チェックする。
- **入力:** 真の効果が `arm1=+1, arm2=+2` になるように自前で生成したおもちゃデータ。
- **出力 / 副作用:** `tau2_hat > tau1_hat` かつ、値が極端におかしくないことを `assert` で検証する。
- **なぜこのアプローチか:** 厳密な一致は目指さず、「効果の大小関係」が保たれていることだけを見る方が、実務で壊れにくく、テストも安定する。
- **実行方法:**

```
cd backend
pytest tests/v2/test_multi_arm_dr.py -q
```

Expert insight (Google/Meta/NASA レベル)

多アーム系のテストは「真値との誤差」よりも **ランク順序の検証** に寄せると壊れにくいです。
 広告配信でも「どのクリエイティブが上位か」が重要で、数値の誤差は後段でキャリブレーションする発想が一般的です。

4. モジュール B - Experiment Orchestrator & Bandit Control Tower の確認

4.1 API が立っているか

- `POST /api/v1/experiments`
- `GET /api/v1/experiments/{id}/allocation`

が FastAPI のルーターに存在するかを確認し、
 curl で叩いて 200 が返るかを見る。

```
# 実サーバー起動後
curl -X POST http://localhost:8000/api/v1/experiments \
  -H 'Content-Type: application/json' \
  -d '{"experiment_id": "exp_demo", "arms": ["A", "B", "C"], "target_metric": "delta_yen"}'

curl http://localhost:8000/api/v1/experiments/exp_demo/allocation
```

4.2 テスト : Thompson Sampling が「マシン」アームに寄るか

```
# backend/tests/v2/test_thompson_bandit.py
from backend.engine.bandits.thompson import BernoulliThompsonBandit
```

```
def test_thompson_prefers_best_arm():
    bandit = BernoulliThompsonBandit(["A", "B", "C"])

    true_p = {"A": 0.02, "B": 0.05, "C": 0.01}

    # 擬似的に 5000 ステップ回す
    history = []
    for _ in range(5000):
        alloc = bandit.sample_allocation()
        # 1 ユーザーだけ配信したと仮定
        arm = max(alloc, key=alloc.get)
        reward = 1 if np.random.rand() < true_p[arm] else 0
        bandit.update([(arm, reward)])
        history.append(arm)

    # 最後の 500 ステップで B が 50% 以上なら OK とする
    tail = history[-500:]
    share_B = sum(a == "B" for a in tail) / len(tail)
    assert share_B > 0.5
```

What this does

- **ゴール:** Thompson Sampling 実装が、真の成功率が一番高いアーム（ここでは B）を好むように学習できているかを確認する。
- **入力:** 真の CTR を {A:2%, B:5%, C:1%} に固定したシミュレーション。
- **出力:** 最終フェーズで B が 50%以上選ばれていればテスト成功。
- **なぜこのアプローチか:** 理論的な証明ではなく、実務的に十分な学習挙動があるかを Monte Carlo 的に確認するため。
- **実行方法:**

```
cd backend
pytest tests/v2/test_thompson_bandit.py -q
```

Expert insight

オンライン最適化アルゴリズムは「オフラインログでは良さそうなのに本番で暴走する」ことが多いので、シミュレーションで挙動を見ておくことが、世界トップのプロダクトでもほぼ必須です。

5. モジュール C - Growth & LTV Planning Studio の確認

5.1 CLV 推定器のテスト

V2 仕様では Survival × 割引現在価値で CLV を出す方針が書かれています [🔗 V2](#)

```
# backend/tests/v2/test_ltv_estimator.py
import pandas as pd
from backend.engine.ltv.ltv_estimator import SimpleCLVEstimator

def test_clv_treated_higher_than_control():
    # treated の方が少し高い LTV になるようなイベント
    df = pd.DataFrame([
        # t=1
        {"user_id": 1, "t": 1, "revenue": 100, "treated": False},
        {"user_id": 2, "t": 1, "revenue": 100, "treated": True},
        # t=2
        {"user_id": 1, "t": 2, "revenue": 0, "treated": False},
        {"user_id": 2, "t": 2, "revenue": 200, "treated": True},
    ])

    est = SimpleCLVEstimator(discount_rate=0.0)
    est.fit(df)
    clv_control = est.clv(treated=False)
    clv_treated = est.clv(treated=True)

    assert clv_treated > clv_control
    assert clv_control > 0
```

What this does

- **ゴール:** `SimpleCLVEstimator` が、treated 側に追加の売上があると LTV を高く出せるかどうかを確認する。

- 入力: treated ユーザだけ 2 期目に追加課金がある極小データ。
- 出力: `clv_treated > clv_control` であることの確認。
- 実行方法:

```
cd backend
pytest tests/v2/test_ltv_estimator.py -q
```

Expert insight

LTV は値そのものよりも「**treated と control の差が方向として正しいか**」が重要です。
絶対値はビジネス側の前提（割引率・期間）で大きく変わるため、テストでは差分の符号に集中します。

6. モジュール D - Risk & Governance Center の確認

6.1 バイアス検知のテスト

仕様では、男女間の uplift 差分などから Disparity を計算する関数が例示されています [V2](#)

```
# backend/tests/v2/test_governance_disparity.py
import pandas as pd
from backend.engine.governance.fairness import uplift_disparity, check_uplift_gate

def test_uplift_disparity_flags_bias():
    df = pd.DataFrame([
        {"gender": "M", "delta_yen": 100},
        {"gender": "M", "delta_yen": 120},
        {"gender": "F", "delta_yen": 10},
        {"gender": "F", "delta_yen": 15},
    ])
    d = uplift_disparity(df, sensitive_col="gender", group_a="M", group_b="F")
    assert d > 50 # 明確に大きな差が出る

    status = check_uplift_gate(d, threshold=30)
    assert status == "FAIL"
```

What this does

- ゴール: uplift の男女差が大きい時に Governance が「FAIL」を返すことを検証する。
- 入力: 男性だけ Δ¥ が大きく、女性はほぼゼロという極端なサンプル。
- 出力: `uplift_disparity` が 50 以上になり、`check_uplift_gate` が "FAIL" を返すこと。
- 実行方法:

```
cd backend
pytest tests/v2/test_governance_disparity.py -q
```

Expert insight

ガバナンス系は「しきい値のチューニング」はあとで変わるので、
関数の形と方向（差が大きいほど **Fail** 寄り）が正しいか だけをテストで固定するのがよくあるパターンです。

7. UI / 可視化の確認フロー

コードが通ったら、**1 つのサンプル CSV / イベントログを決めて E2E で確認**します。

1. **Causal Design**
 - Treatment Type = Multi-Armed を選ぶ
 - サンプル多アーム CSV をアップ → 各 arm の Δ¥ 棒グラフと Pairwise Comparison が出るか
2. **Experiment Studio (Orchestrator)**
 - 新しい experiment を UI から定義
 - Allocation Panel に A/B/C の比率が見え、モックイベントを投げると数字が変化するか
3. **Growth Studio (LTV)**
 - 購入履歴 CSV をアップ → CLV Summary / Cohort Retention Plot が描画されるか
4. **Governance**
 - わざと gender 偏ったサンプルを流し、該当ポリシーが「Fail」になり Export Gate からエクスポート不可になるか

この 4 ステップを「V2 受け入れチェックリスト」として README か Notion に 1 ページでまとめておくと、社内共有・テストも楽になります。

8. まとめ

- 確認方法の本質は、「V2.pdf の各モジュールに 最低 1 つ以上のユニットテスト + 1 つの E2E シナリオ を紐付ける」ことです。
- 上に書いたテストファイルを（必要に応じて修正しつつ）置けば、`pytest tests/v2 -q` が「V2 全体が数学的に破綻していないか」のスモークテストになります。
- UI 側は 1 つのサンプルデータセットで 4 画面（Causal / Experiment / Growth / Governance）を順に回るだけのマニュアル手順を作る。これを一度作っておけば、「これらができているか？」は常に `pytest` の結果と 4 つの画面スクリーンショットで説明できるようになります。